Lezione 8

Argomenti	Stringhe e Tuple
Data	@November 10, 2025

📘 Lezione 8 – Stringhe, Liste e Tuple

👩‍🏫 **Obiettivo della lezione:** lavorare con le **sequenze** in Python usando ciò che è già stato visto:

- stringhe (immutabili)
- liste (mutabili)
- tuple (immutabili)

▶️ Ogni esercizio contiene **consegna**, **input/output d'esempio** e **soluzione**.

⏱️ Durata stimata: **1h 30'**

📚 Struttura:

- **6 esercizi sulle stringhe**
- **6 esercizi sulle liste**
- **2 esercizi sulle tuple**

✏️ STRINGHE – Ripasso rapido

- Una **stringa è una sequenza immutabile** di caratteri.
- Si indicizza come una lista: `s[0]`, `s[-1]`.
- Funzioni/metodi utili: `len()`, `in`, `replace()`, `split()`.

🟢 STRINGHE – 6 esercizi

🟢 Facili

🔷 Esercizio 1 – Conta le vocali

📘 **Consegna:** Definisci la funzione `conta_vocali(s)` che restituisce quante **vocali** sono presenti nella stringa `s`.

📩 **INPUT (esempio):**

```
Matematica
```

📤 **OUTPUT (esempio):**

```
5
```

✅ **Soluzione:**

```
def conta_vocali(s):
    s = s.lower()
    vocali = "aeiou"
    cont = 0
    i = 0
    while i < len(s):
        if s[i] in vocali:
            cont += 1
        i += 1
    return cont
```

🔷 Esercizio 2 – Carattere più frequente

📘 **Consegna:** Definisci la funzione `carattere_piu_frequente(s)` che restituisce una tupla con il carattere più frequente e il suo numero di occorrenze.

📩 **INPUT (esempio):**

```
abbccc
```

📤 **OUTPUT (esempio):**

```
('c', 3)
```

✅ **Soluzione:**

```
def carattere_piu_frequente(s):
    s = s.replace(" ", "")
    maxc = None
    maxf = 0
    i = 0
    while i < len(s):
        ch = s[i]
        f = s.count(ch)
        if f > maxf:
            maxf = f
            maxc = ch
        i += 1
    return maxc, maxf
```

🟡 Medi

🔷 Esercizio 3 – Palindroma

📘 **Consegna:** Scrivi la funzione `is_palindroma(s)` che restituisce `True` se la stringa è **palindroma** (uguale letta da sinistra e da destra).

📥 **INPUT (esempio):**

```
Anna
```

📤 **OUTPUT (esempio):**

```
True
```

✅ **Soluzione:**

```
def is_palindroma(s):
    s = s.lower()
    return s == s[::-1]
```

🔷 Esercizio 4 – Somma delle cifre nella stringa

📘 **Consegna:** Definisci `somma_cifre(s)` che restituisce la somma delle **cifre numeriche** presenti nella stringa.

📩 **INPUT (esempio):**

```
a1b2c3
```

📤 **OUTPUT (esempio):**

```
6
```

✅ **Soluzione:**

```
def somma_cifre(s):
    tot = 0
    i = 0
    while i < len(s):
        if s[i].isdigit():
            tot += int(s[i])
        i += 1
    return tot
```

🔷 Esercizio 5 – Censura parola

📘 **Consegna:** Definisci `censura(frase, parola)` che sostituisce ogni occorrenza di `parola` con `***`.

📩 **INPUT (esempio):**

```
frase = "oggi piove"
parola = "piove"
```

📤 **OUTPUT (esempio):**

```
oggi ***
```

✅ **Soluzione:**

```
def censura(frase, parola):
    return frase.replace(parola, "***")
```

🔴 Difficile

🔷 Esercizio 6 – Parola più lunga

📘 **Consegna:** Data una frase, restituisci la **parola più lunga**.

📩 **INPUT (esempio):**

```
la matematica e bella
```

📤 **OUTPUT (esempio):**

```
matematica
```

✅ **Soluzione:**

```
def parola_piu_lunga(frase):
    parole = frase.split()
    maxp = ""
    i = 0
    while i < len(parole):
        if len(parole[i]) > len(maxp):
            maxp = parole[i]
        i += 1
    return maxp
```

🔵 LISTE – Ripasso rapido

- Liste = sequenze **mutabili**.
- Metodi utili: `.append()`, `.sort()`, `.remove()`.

🟢 LISTE – 6 esercizi

🟢 Facili

🔷 Esercizio 7 – Somma di 5 numeri (facile)

📘 **Consegna:** Scrivi la funzione `leggi_e_somma()` che chiede **5 numeri** all'utente (uno per riga), li inserisce in una lista e restituisce la loro somma.

📩 **INPUT (esempio):**

```
1
2
3
4
5
```

📤 **OUTPUT (esempio):**

```
15.0
```

✅ **Soluzione:**

```
def leggi_e_somma():
    lst = []
    i = 0
    while i < 5:
        lst.append(float(input("Numero: ")))
        i += 1
    return sum(lst)

print(leggi_e_somma())
```

🧠 *Esercizio semplice per introdurre* `append()` *e* `sum()`.

🔷 Esercizio 8 – Filtra pari

📘 **Consegna:** Definisci `filtra_pari(lista)` che prende una lista di interi e restituisce una nuova lista contenente **solo i numeri pari**.

📩 **INPUT (esempio):**

```
[1, 2, 3, 4, 5, 6]
```

📤 **OUTPUT (esempio):**

```
[2, 4, 6]
```

✅ **Soluzione:**

```
def filtra_pari(lista):
    res = []
    i = 0
    while i < len(lista):
        if lista[i] % 2 == 0:
            res.append(lista[i])
        i += 1
    return res

print(filtra_pari([1, 2, 3, 4, 5, 6]))
```

🧠 *Esercizio per praticare l'accesso per indice e la costruzione di una nuova lista.*

🟡 Medi

🔷 Esercizio 9 – Rimuovi duplicati mantenendo ordine

📘 **Consegna:** Definisci `rimuovi_duplicati(lista)` che, data una lista, restituisce una versione senza duplicati **mantenendo l'ordine di prima occorrenza**.

📥 **INPUT (esempio):**

```
[1, 4, 2, 1, 4, 5]
```

📤 **OUTPUT (esempio):**

```
[1, 4, 2, 5]
```

✅ **Soluzione:**

```
def rimuovi_duplicati(lista):
    seen = []
    i = 0
    while i < len(lista):
```

```
        if lista[i] not in seen:
            seen.append(lista[i])
        i += 1
    return seen

print(rimuovi_duplicati([1, 4, 2, 1, 4, 5]))
```

🧠 *Mostra come usare una lista di supporto per preservare l'ordine.*

🔷 Esercizio 10 – Media di una lista

📘 **Consegna:** Definisci `media_lista(lista)` che calcola la media aritmetica degli elementi di `lista`. Se la lista è vuota restituisce `None`.

📥 **INPUT (esempio):**

```
[2, 4, 6]
```

📤 **OUTPUT (esempio):**

```
4.0
```

✅ **Soluzione:**

```
def media_lista(lista):
    if len(lista) == 0:
        return None
    return sum(lista) / len(lista)

print(media_lista([2, 4, 6]))
```

🧠 *Esercizio per consolidare* `sum()` *e* `len()` *e la gestione dei casi limite.*

🔷 Esercizio 11 – Ordina nomi

📘 **Consegna:** Leggi una riga di input contenente nomi separati da spazi, mettili in una lista e stampa la lista ordinata in ordine alfabetico.

📥 **INPUT (esempio):**

```
marco anna luca
```

📤 **OUTPUT (esempio):**

```
['anna', 'luca', 'marco']
```

✅ **Soluzione:**

```
s = input("Nomi separati da spazio: ")
nomi = s.split()
nomi.sort()
print(nomi)
```

🧠 *Introduzione all'uso di* `split()` *per creare liste da stringhe e* `.sort()` *per ordinarle.*

🔴 Difficile

🔷 Esercizio 12 – Massimo scarto assoluto

📘 **Consegna:** Definisci `scarto_max_min(lista)` che restituisce la differenza tra massimo e minimo elemento della lista. Se la lista è vuota restituisce `None`.

📥 **INPUT (esempio):**

```
[1, 43, 5, 100]
```

📤 **OUTPUT (esempio):**

```
99
```

✅ **Soluzione:**

```
def scarto_max_min(lista):
    if not lista:
        return None
    return max(lista) - min(lista)
```

```
print(scarto_max_min([1, 43, 5, 100]))
```

🧠 *Esercizio per consolidare* `max()` *e* `min()` *e pensare ai casi limite.*

🟣 TUPLE – Ripasso

- Simili a liste, ma **immutabili**.

🟣 TUPLE – 2 esercizi

✅ Facile

🔷 Esercizio 13 – Coordinate

📘 **Consegna:** Leggi le coordinate (x, y) di un punto nel piano e memorizzale in una **tupla**.

📥 **INPUT (esempio):**

```
3
5
```

📤 **OUTPUT (esempio):**

```
(3.0, 5.0)
```

✅ **Soluzione:**

```
x = float(input("x: "))
y = float(input("y: "))
t = (x, y)
print(t)
```

🟡 Medio

🔷 Esercizio 14 – Distanza dall'origine

📘 **Consegna:** Definisci una funzione `distanza(t)` che riceve una tupla contenente `(x, y)` e calcola la distanza del punto **dall'origine** usando il teorema di Pitagora.

📩 **INPUT (esempio):**

```
t = (3, 4)
```

📤 **OUTPUT (esempio):**

```
5.0
```

✅ **Soluzione:**

```
from math import sqrt

def distanza(t):
    return sqrt(t[0]**2 + t[1]**2)
```

📅 Data: 10 novembre 2025

👨‍🏫 Tutor: Francesco Caligiuri

💻 Tema: Stringhe, Liste, Tuple

📽️ Versione per proiezione con soluzioni